PHASE 4: SYSTEM IMPLEMENTATION

4.1 Introduction

This phase describes the implementation of the Learning Management System (LMS) using C++17 and SQLite. It is an offline console application for a secondary school with four user roles: Administrator, Lecturer, Head of Department (HoD) and Student.

The implementation follows the layered architecture designed in Phase 3. It supports course management, assignments, submissions, late penalties, individual extensions, marking, moderation, release of marks and an audit log.

Git repository: https://github.com/makoakhotso-oss/CS4433-MODERATED-LMS-PROJECT

4.2 Development Tools

Tool	Purpose
C++17	Developing the application and business rules
SQLite	Storing system records in a local database
CMake	Configuring and building the project
libsodium	Password hashing using Argon2id
GoogleTest	Automated testing
Git	Version control and source code management

4.3 Implementation Structure

The source code is organised into the src folder, with header files in include/lms. It is divided into six main parts:

- **ui:** Console menus and user input.
- **services:** Business rules for authentication, access control, users, courses, assignments, submissions, marking, moderation and results.
- **data:** Database operations, repository interfaces, SQLite repositories and submission file storage.
- **domain:** Core objects such as User, Course, Assignment, Submission and MarkRecord.
- **security:** Password hashing through PasswordHasher.
- **common:** Shared validation, time, error-handling and logging functions.

Services use repository interfaces rather than accessing SQLite directly. The database schema is stored in sql/schema.sql and included in the build through CMake.

4.4 Functional Implementation

The main system functions are implemented through the following components:

Function	Implementation
Login and passwords	AuthService and PasswordHasher
Users and departments	UserService
Courses and enrolment	CourseService
Role permissions	AccessControl and CourseAccess

Assignments and extensions	AssignmentService
Submission and resubmission	SubmissionService and FileStorage
Late penalties	StandardLatePenaltyPolicy
Marking and feedback	MarkingService
Moderation and release	ModerationService
Student results	ResultsService
Mark audit history	AuditLog and mark_audit_log table

User roles: The Administrator manages users, departments, courses, lecturer assignments and student enrolment. The Lecturer manages assignments, grants individual extensions, marks work and sends marks for moderation. The HoD reviews marks for courses in their department and either approves them or returns them to the lecturer. Students view enrolled courses, submit work and see marks after release. Access control is checked before protected actions.

Submissions: Each submission attempt is stored as a separate record, with optional comments. Students can resubmit while the applicable deadline has not passed, even if an earlier attempt has been marked. Previous attempts and their records are retained, while the newest attempt becomes the current one.

Late penalties: Work submitted up to three days late loses 10% of the available mark for each day late. For example, an assignment out of 20 loses 2 marks after one day, so a raw mark of 18 becomes 16. A first submission more than three days late is saved with a zero mark, while a late resubmission is refused. An approved individual extension changes the applicable deadline.

Marking and release: Lecturers record marks and feedback before sending them for moderation. The HoD approves the marks, which also releases them to students, or returns them to the lecturer for changes. The database prevents release without an approval record. Students cannot view unreleased marks.

4.5 Object-Oriented Design

The implementation uses object-oriented programming principles and design patterns:

- **Inheritance and polymorphism:** User is an abstract class. Administrator, Lecturer, HeadOfDepartment and Student inherit from it and provide their own permissions through a virtual function.
- **Strategy pattern:** ILatePenaltyPolicy and StandardLatePenaltyPolicy separate late-penalty rules from SubmissionService.
- **Repository pattern:** Interfaces such as ISubmissionRepository and IMarkRepository separate business logic from database operations. SQLite repository classes implement these interfaces.
- **Factory pattern:** UserFactory creates the appropriate user object for each role.
- **SOLID principles:** Each service has a specific responsibility, and services depend on interfaces rather than concrete SQLite classes.

4.6 Database and Persistence

The database contains ten tables:

No.	Table	Purpose
1	departments	Stores department records

2	users	Stores user accounts and role information
3	courses	Stores course information
4	enrollments	Links students to courses
5	assignments	Stores assignment details and deadlines
6	extensions	Stores individual deadline extensions
7	submissions	Stores submission attempts and comments
8	marks	Stores marks, feedback and moderation state
9	moderation_decisions	Stores moderation decisions
10	mark_audit_log	Records mark entries and changes

The database also uses indexes, foreign keys, unique constraints, check constraints and triggers. These enforce rules such as preventing marks above the assignment maximum, requiring enrolment before submission and preventing release without approval. Audit records are protected against modification or deletion.

All SQL operations go through the data layer and use prepared statements with bound values. Submission and moderation changes use database transactions. Foreign-key enforcement is enabled for each connection.

Data is stored in a local SQLite database file and remains available after the application closes.

4.7 Security and Error Handling

1. **Passwords:** Passwords are hashed using Argon2id through libsodium. Only salted password hashes are stored. Login attempts are temporarily locked after repeated failures.
2. **Access control:** Each protected action requires permission. HoDs can moderate courses in their own departments, while lecturers can mark work for their own courses.
3. **Input validation:** The system checks text length, mark ranges, reasons and file sizes before saving.
4. **Audit log:** Mark entries and changes record the user, date and time, old mark, new mark and reason. A reason is required when an existing mark is changed.
5. **Error handling:** Exceptions handle invalid input and rule violations to prevent incorrect operations from being processed.

Only passwords are hashed. The database file itself is not encrypted.

4.8 Differences from the Design

Some differences between earlier design documents and the final implementation are documented in docs/AS_BUILT_DEVIATIONS.md. The main differences are:

- Very late first submissions are saved with a zero mark, while late resubmissions are refused.
- Lecturers can create their own courses. Administrators can also create courses, assign lecturers and enrol students.
- A departments table was added to support department-based HoD access.
- A moderation_state column was added to the marks table.
- A comments column was added to submissions.

- The HoD can approve marks or return them to the lecturer. Student resubmission after moderation rejection was not implemented.
- Operations are handled by service classes rather than domain classes, and additional repository interfaces were introduce